Clase 10 (Unidad 4) — Broadcast, presencia y escalabilidad con Redis

Duración total: 4 horas | Proyecto: notificaciones en grupo, presencia online, WebSocket autenticado y channel layer con Redis

Cómo leer esta guía

Cada parte combina dos tipos de contenido:

- **Conceptos** (qué es y por qué): la base teórica de la Unidad 4, con tablas y diagramas. Se pueden leer antes de la clase.
- **Práctica** (cómo): los pasos con código, que aplican esos conceptos a TaskFlow.

Al final hay **preguntas de repaso** para autoevaluarse.

Parte 0 — Antes de empezar (5 min)

Parados en `taskflow/`, entorno virtual activo. Al cierre de la Clase 9 tenemos un WebSocket funcionando en modo "echo", pero abierto a cualquiera. Hoy lo transformamos en un sistema real de notificaciones compartidas entre varios usuarios conectados al mismo proyecto, y lo protegemos con las mismas reglas de la Unidad 3.

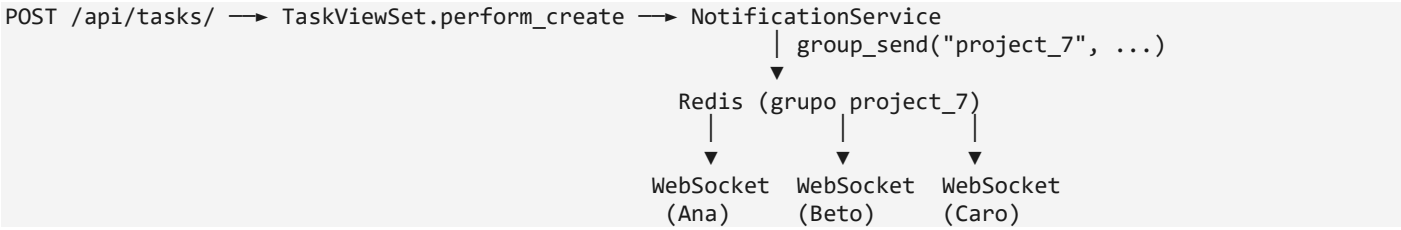
Parte 1 — Repaso del cuestionario y arquitectura orientada a eventos (30 min)

1.1 Puesta en común

Repasamos entre todos las respuestas al cuestionario de la Unidad 4 (handshake WebSocket, Redis vs. Kafka, desacoplamiento en EDA, límites de SSE, autenticación stateful). El docente aclara dudas antes de seguir con la práctica.

1.2 Arquitectura orientada a eventos (EDA)

Hoy vamos a introducir el patrón **Productor → Broker → Consumidor**: cuando pasa algo relevante en TaskFlow (se crea una tarea), un "productor" (nuestra vista de DRF) publica un evento en un "broker" (Redis), y todos los "consumidores" suscriptos (cada conexión WebSocket abierta a ese proyecto) lo reciben. Ningún componente conoce directamente al otro: ni la vista sabe cuántas conexiones WebSocket hay abiertas, ni el consumer sabe qué vista generó el evento. Eso es el **desacoplamiento**, la ventaja central de este patrón frente a que la vista tuviera que "avisarle a cada conexión" manualmente.



Concepto: eventos, comandos y mensajes. En una arquitectura orientada a eventos conviene distinguir:

- Un **comando** pide que algo pase: "creá esta tarea" (`POST /api/tasks/`). Tiene un destinatario y puede ser rechazado.
- Un **evento** informa que algo **ya pasó**: "se creó la tarea 42" (`task_created`). Se nombra en pasado, no se puede rechazar, y quien lo publica no sabe quién lo va a escuchar.

Concepto: dos formas de repartir mensajes.

Modelo	Cómo reparte	Ejemplo
Publicar/suscribir (*pub/sub*)	Cada mensaje llega a **todos** los suscriptos	Nuestros grupos: todos los conectados al proyecto reciben el aviso
Cola de mensajes (punto a punto)	Cada mensaje lo procesa **uno solo** de los consumidores	Una cola de "enviar email": varios trabajadores se reparten los pendientes

Concepto: garantías de entrega. ¿Qué pasa si un mensaje se pierde?

Garantía	Significa	Ejemplo
Como máximo una vez (*at-most-once*)	Puede perderse, nunca se duplica	Channels: si un usuario estaba desconectado en ese momento, no recibe el aviso
Al menos una vez (*at-least-once*)	Nunca se pierde, pero puede llegar repetido	RabbitMQ, Kafka, Redis Streams, con confirmación
Exactamente una vez	Ni se pierde ni se repite	Muy difícil; en la práctica se logra con "al menos una vez" más consumidores idempotentes (Clase 5)

El channel layer de Channels **no es una cola duradera**: está pensado para avisos en vivo. Por eso los datos "de verdad" siguen en la base, y el cliente, al reconectarse, vuelve a pedirlos por la API REST (Parte 3.5). A esa idea (el sistema se pone de acuerdo "un rato después", no al instante) se la llama **consistencia eventual**.

Concepto: brokers más usados. Esta es la base para responder la pregunta 3 del cuestionario:

Broker	Modelo	Persistencia	Uso típico
Redis pub/sub	Pub/sub	No	Avisos en vivo, channel layers (nuestro caso)
Redis Streams	Registro de eventos	Sí	Colas simples y livianas
RabbitMQ	Colas y pub/sub	Sí	Tareas en segundo plano entre servicios
Apache Kafka	Registro distribuido de eventos	Sí, por días o meses	Grandes volúmenes, analítica, sistemas que "releen" la historia

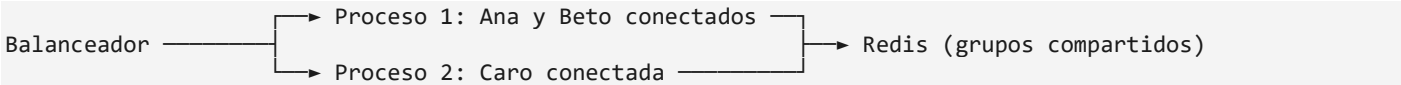
Parte 2 — Redis como Channel Layer (70 min)

2.1 Por qué no alcanza el layer en memoria

En la Clase 9 configuramos `InMemoryChannelLayer`: el pizarrón de grupos vive en la memoria del proceso de Django. Eso es válido solo mientras haya **un único proceso** de servidor corriendo. Apenas escalemos a más de un proceso (algo que se hace en producción, Clase 12), cada proceso tendría su propia memoria aislada, y los grupos de un proceso no se enterarían de los mensajes publicados desde otro. Redis resuelve esto siendo un almacén **externo y compartido**, al que todos los procesos del servidor se conectan por igual.

Concepto: escalar vertical u horizontalmente.

- **Escalar verticalmente** es darle más recursos a la misma máquina (más memoria, más procesador). Es simple, pero tiene un techo.
- **Escalar horizontalmente** es sumar **más copias** de la aplicación detrás de un **balanceador de carga**, que reparte las conexiones entre ellas. No tiene techo, pero exige que las copias no guarden estado propio, o que lo compartan.



Sin Redis, un aviso publicado en el Proceso 1 nunca llegaría a Caro. Con Redis, todos los procesos publican y escuchan en el mismo lugar. Es la misma razón por la que JWT (Clase 6) escala mejor que las sesiones: el estado compartido va a un servicio externo, no a la memoria de cada copia.

2.2 Instalación

Primero necesitamos Redis corriendo como un servicio aparte (no es una librería de Python: es un programa independiente, **gratuito y de código abierto**). Formalizamos Docker recién en la Clase 12, así que acá damos alternativas equivalentes por sistema operativo. Elijan la que les resulte más simple:

Opción A — Linux: se instala directamente con el gestor de paquetes de la distribución:

```
sudo apt update && sudo apt install redis-server
sudo service redis-server start
```

Opción B — Mac: con Homebrew (`brew.sh`, gratuito):

```
brew install redis
brew services start redis
```

Opción C — Windows: Redis no tiene un instalador oficial para Windows. La forma más simple y recomendada es **WSL2** (Windows Subsystem for Linux, gratuito, incluido en Windows 10/11): se instala una vez con `wsl --install` desde PowerShell **como administrador** (reinicia la computadora al finalizar), y después, dentro de la terminal de Ubuntu que WSL habilita, se sigue exactamente la Opción A de Linux. Django, que corre en Windows, llega a ese Redis igual por `127.0.0.1`.

Opción D — cualquier sistema operativo, con Docker (si ya lo tienen instalado):

```
docker run -d --name taskflow-redis -p 6379:6379 redis:7
```

`docker run` descarga (si no la tienen) la imagen oficial `redis:7` y la ejecuta. `-d` (*detached*) la corre en segundo plano; `--name taskflow-redis` le da un nombre para poder referenciarla después (`docker stop taskflow-redis`, `docker start taskflow-redis`); `-p 6379:6379` conecta el puerto 6379 de nuestra computadora con el puerto 6379 dentro del contenedor, que es el puerto estándar en el que Redis escucha. Retomamos Docker en detalle en la Clase 12.

Con cualquiera de las cuatro opciones, verifiquen que Redis quedó corriendo:

```
redis-cli ping
```

Debería responder `PONG`. (Con la Opción D, si no tienen `redis-cli` instalado: `docker exec taskflow-redis redis-cli ping`.)

Ahora sí, la librería de Python que conecta Channels con Redis, parados en `taskflow/` con el entorno virtual activo:

```
pip install channels-redis
pip freeze > requirements.txt
```

2.3 Configuración

En `taskflow/config/settings.py`, **reemplacen** el bloque `CHANNEL_LAYERS` de la Clase 9 por:

```
CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {
            "hosts": [os.getenv("REDIS_URL", "redis://127.0.0.1:6379")],
        },
    },
}
```

`hosts` es la lista de servidores Redis a los que se conecta Channels. La dirección se lee de la variable de entorno `REDIS_URL` y, si no existe, usa `redis://127.0.0.1:6379` (Redis en la misma computadora). Hoy no hace falta agregar nada al `.env`; en la Clase 12, dentro de Docker, Redis va a estar en otra dirección y solo vamos a cambiar esa variable, sin tocar el código.

Reinicien el servidor Django (`Ctrl+C` y de nuevo `python manage.py runserver`) y repitan la prueba de conexión de la Clase 9 (Parte 3.3): debería seguir funcionando exactamente igual desde afuera, pero ahora el pizarrón de grupos vive en Redis en vez de en la memoria del proceso de Django.

Parte 3 — Broadcast de eventos de negocio, seguridad y presencia (95 min)

3.1 Emitir un evento desde la API REST

Queremos que, cuando alguien cree una tarea vía `POST /api/tasks/` (el endpoint REST que construimos en la Clase 4), todos los usuarios conectados por WebSocket a ese proyecto reciban la notificación en tiempo real, sin refrescar la página.

Concepto: qué viaja en un evento. Hay dos estilos:

- **Evento liviano:** solo el aviso y los identificadores (`{"event": "task_created", "task_id": 42}`). El cliente pide los detalles por la API REST, que aplica todos los permisos.
- **Evento completo:** todos los datos necesarios (`title`, `status`, `priority`...). El cliente no necesita hacer otra petición, pero hay que cuidar no enviar datos que alguno de los destinatarios no debería ver.

Nuestro `task_created` es casi liviano: `id` y título. Una regla de seguridad importante: el evento se manda a un grupo cuyos integrantes ya pasaron por el control de acceso (Parte 3.3), así que solo lo reciben miembros del proyecto.

Concepto: publicar después de guardar. Un evento anuncia algo que **ya pasó**, así que solo debe publicarse cuando el dato está efectivamente guardado. Si la operación estuviera dentro de una transacción (Clase 7) y la transacción se deshiciera después de publicar, los clientes recibirían el aviso de una tarea que no existe. Django ofrece `transaction.on_commit(función)` para ejecutar algo recién cuando la transacción se confirma. En nuestro `perform_create` no hay una transacción abierta, así que `serializer.save()` ya guardó cuando publicamos.

Práctica.

En `taskflow/core/services.py`, agreguen los imports arriba y la nueva clase al final:

```
import logging

from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer

logger = logging.getLogger(__name__)


class NotificationService:
    @staticmethod
    def notify_task_created(task):
        NotificationService._send(task.project_id, {
            "event": "task_created",
            "task_id": task.id,
            "title": task.title,
        })

    @staticmethod
    def _send(project_id, payload):
        channel_layer = get_channel_layer()
        try:
            async_to_sync(channel_layer.group_send)(
                f"project_{project_id}",
                {
                    "type": "task_notification",  # coincide con el método del consumer
                    "payload": payload,
                },
            )
        except Exception:
            # Si Redis está caído, la tarea igual se guardó: solo avisamos en el log.
            logger.exception("No se pudo enviar la notificación en tiempo real")
```

Paso a paso:

- `get_channel_layer()` obtiene el channel layer configurado (el de Redis, Parte 2.3).
- `channel_layer.group_send(...)` es una función **asincrónica** (`async`), pero `perform_create` (donde la vamos a llamar) es código **sincrónico**. Por eso la envolvemos con `async_to_sync(...)`, el puente entre ambos mundos que provee `asgiref` (una dependencia que ya vino con Django).
- El primer argumento de `group_send` es el **nombre del grupo** destinatario. Tiene que coincidir exactamente con el que usa el consumer en `connect` (Clase 9): `f"project_{self.project_id}"`.

- El segundo argumento es un diccionario con una clave obligatoria, `"type"`. Su valor (`"task_notification"`) tiene que ser **exactamente** el nombre de un método del consumer: Channels lo invoca automáticamente, por ese nombre, en cada conexión del grupo. `"payload"` es un nombre elegido por nosotros para los datos del evento.
- `_send` empieza con guion bajo por convención de Python: indica que es de uso interno de la clase. Separarlo nos permite agregar más eventos (`task_updated`, `task_completed`) reutilizando el envío.
- El `try/except` sigue la misma idea que la API de feriados de la Clase 5: si Redis está caído, **la tarea ya se guardó** en la base de datos. No tiene sentido responderle un error 500 al usuario por no poder enviar una notificación: lo dejamos registrado en el log y seguimos. `logger.exception(...)` guarda el mensaje junto con el detalle completo del error.

En `taskflow/core/views.py`, dentro de `TaskViewSet`, actualicen `perform_create` (el de la Clase 7) para que, además de verificar el rol, notifique. Sumen `NotificationService` al import de servicios:

```
from .services import MembershipService, NotificationService, TaskService


    def perform_create(self, serializer):
        self.check_project_admin(serializer.validated_data["project"])
        task = serializer.save(owner=self.request.user)
        NotificationService.notify_task_created(task)
```

`serializer.save(...)` devuelve el objeto recién guardado: lo usamos para armar la notificación con su `id` y su `title`.

3.2 Autenticar el WebSocket con JWT

Recordatorio de la Clase 9: hoy **cualquiera** puede conectarse a `ws/projects/<id>/` y escuchar las notificaciones de cualquier proyecto. Es exactamente el *Broken Access Control* que vimos en la Clase 8. Tenemos que aplicarle al WebSocket las mismas reglas que a la API REST: usuario autenticado **y** miembro del proyecto.

El problema: la API REST reconoce al usuario por el header `Authorization: Bearer ...`, pero el `WebSocket` del navegador **no permite** agregar headers propios al conectarse. La solución habitual es mandar el token como parámetro en la URL: `ws://.../ws/projects/7/?token=<access>`.

Creen el archivo `taskflow/core/ws_auth.py`:

```
from urllib.parse import parse_qs

from channels.db import database_sync_to_async
from channels.middleware import BaseMiddleware
from django.contrib.auth import get_user_model
from django.contrib.auth.models import AnonymousUser
from rest_framework_simplejwt.exceptions import TokenError
from rest_framework_simplejwt.tokens import AccessToken


@database_sync_to_async
def get_user_from_token(token):
    User = get_user_model()
    try:
        access = AccessToken(token)
        return User.objects.get(id=access["user_id"])
    except (TokenError, KeyError, User.DoesNotExist):
        return AnonymousUser()


class JWTAuthMiddleware(BaseMiddleware):
    """Lee ?token=<access> de la URL del WebSocket y completa scope["user"]."""

    async def __call__(self, scope, receive, send):
        query = parse_qs(scope.get("query_string", b"").decode())
        token = query.get("token", [None])[0]
        if token:
            scope["user"] = await get_user_from_token(token)
        return await super().__call__(scope, receive, send)
```

- Un **middleware de Channels** cumple el mismo rol que los middleware de Django (Clase 8): se ejecuta antes de que la conexión llegue al consumer, y puede agregar información al `scope`.
- `scope["query_string"]` es la parte de la URL después del `?`, en bytes (`b"token=eyJ..."`). `.decode()` la convierte en texto, y `parse_qs` la transforma en un diccionario: `{"token": ["eyJ..."]}` (cada valor es una lista, porque un parámetro podría repetirse; por eso el `[0]`).
- `AccessToken(token)` es la misma validación que usa `JWTAuthentication` en la API REST: verifica la firma y la fecha de vencimiento. Si algo falla, lanza `TokenError`.
- `access["user_id"]` es el `id` del usuario guardado dentro del token (el payload que vimos en jwt.io en la Clase 6).
- `@database_sync_to_async` es el puente que usaron en la tarea de la Clase 9: convierte una función sincrónica que usa el ORM en una que se puede `await`-ear desde código asincrónico.
- Si el token es inválido, dejamos un `AnonymousUser`, el mismo objeto que usa Django para "nadie logueado".

En `taskflow/config/asgi.py`, sumen el import (junto a `import core.routing`) y envuelvan el `URLRouter` con el nuevo middleware:

```
import core.routing
from core.ws_auth import JWTAuthMiddleware

application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AuthMiddlewareStack(
        JWTAuthMiddleware(
            URLRouter(core.routing.websocket_urlpatterns)
        )
    ),
})
```

Concepto: ¿por qué el token va en la URL, y qué alternativas hay?

Forma de autenticar un WebSocket	Ventaja	Desventaja
Token en la URL (`?token=`) — la que usamos	Simple; funciona en cualquier cliente	La URL puede quedar en logs e historiales
Cookie de sesión	Automática en el navegador	Obliga a verificar el origen (ver abajo); no sirve para apps móviles
Primer mensaje	El token no aparece en la URL	Hay que aceptar la conexión antes de saber quién es
Ticket de un solo uso	Pedido por REST, vence en segundos	Una petición extra

Concepto: los WebSockets no están protegidos por CORS. La política del mismo origen (Clase 8) **no** se aplica a los WebSockets: cualquier página puede intentar abrir una conexión a nuestro servidor. Si autenticáramos solo con la cookie de sesión, una página maliciosa podría abrir un WebSocket con la cookie del usuario y leer sus avisos. Ese ataque se llama *Cross-Site WebSocket Hijacking*. Las defensas: verificar el header `Origin` en el handshake (Channels trae `AllowedHostsOriginValidator` para eso) o no depender de la cookie. Nuestro esquema con JWT en la URL no es vulnerable: la página maliciosa no conoce el token.

Las capas se ejecutan de afuera hacia adentro: `AuthMiddlewareStack` intenta reconocer al usuario por la sesión del navegador; después `JWTAuthMiddleware`, si viene un `?token=`, lo reemplaza por el usuario del token; y recién ahí la conexión llega al consumer.

> 💡 **Nota del docente**
> Un token en la URL puede quedar escrito en los logs del servidor y en el historial del navegador. Por eso usamos siempre el `access` (dura 30 minutos), nunca el `refresh`, y en producción siempre sobre `wss://` (cifrado). Existen alternativas más robustas (por ejemplo, un "ticket" de un solo uso pedido por la API REST), pero esta es la más usada y alcanza para la materia.

3.3 El consumer completo: permisos y presencia

Vamos a hacer dos cosas en el consumer: exigir usuario autenticado y miembro del proyecto (Parte 3.2), y llevar un registro de qué usuarios están conectados a cada proyecto (**presencia**), reutilizando el mismo mecanismo de grupos.

En `taskflow/core/consumers.py`, **reemplacen** todo el contenido por:

```
import json

from channels.db import database_sync_to_async
from channels.generic.websocket import AsyncWebsocketConsumer

from .services import MembershipService


class TaskNotificationConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.project_id = self.scope["url_route"]["kwargs"]["project_id"]
        self.group_name = f"project_{self.project_id}"
        self.user = self.scope["user"]

        # Solo usuarios autenticados y miembros del proyecto
        if not self.user.is_authenticated:
            await self.close()
            return
        es_miembro = await database_sync_to_async(MembershipService.is_member)(
            self.user, self.project_id
        )
        if not es_miembro:
            await self.close()
            return

        await self.channel_layer.group_add(self.group_name, self.channel_name)
        await self.accept()
        self.joined = True

        await self.channel_layer.group_send(
            self.group_name,
            {"type": "presence_update", "payload": {"user": self.user.username, "status": "online"}},
        )

    async def disconnect(self, close_code):
        # Si la conexión fue rechazada en connect, nunca entró al grupo
        if not getattr(self, "joined", False):
            return
        await self.channel_layer.group_discard(self.group_name, self.channel_name)
        await self.channel_layer.group_send(
            self.group_name,
            {"type": "presence_update", "payload": {"user": self.user.username, "status": "offline"}},
        )

    async def receive(self, text_data):
        data = json.loads(text_data)
        await self.send(text_data=json.dumps({"echo": data}))

    async def task_notification(self, event):
        await self.send(text_data=json.dumps(event["payload"]))

    async def presence_update(self, event):
        await self.send(text_data=json.dumps(event["payload"]))
```

Concepto: sistemas de presencia. "Presencia" es saber quién está conectado en un momento dado (el puntito verde de las aplicaciones de chat). Parece simple, pero tiene casos difíciles:

- **Varias pestañas o dispositivos:** si Ana tiene dos pestañas abiertas y cierra una, nuestro consumer avisa "offline", aunque siga conectada en la otra. La solución es contar las conexiones por usuario (por ejemplo, en un conjunto de Redis) y avisar "offline" solo cuando llega a cero.
- **Cortes sin aviso:** si se corta la red, el servidor puede tardar en enterarse. Por eso los sistemas de presencia usan **latidos** (*heartbeats*): el cliente manda una señal cada tantos segundos, y si deja de llegar, se lo da por desconectado.
- **Estado del usuario:** además de online u offline, muchas aplicaciones manejan estados como "ausente" o "escribiendo...", que viajan por el mismo mecanismo de grupos.

Nuestra implementación es la versión básica; los dos primeros puntos quedan como desafío.

Qué cambió respecto de la Clase 9:

- `self.user = self.scope["user"]` es el usuario que dejó el middleware de la Parte 3.2 (o `AnonymousUser`).
- Si no está autenticado, o no es miembro del proyecto, `await self.close()` **rechaza** la conexión antes de aceptarla, y `return` corta el método. Del lado del cliente, el handshake falla.
- `MembershipService.is_member` es el mismo método de la Clase 7 que usan los permisos de la API REST: una sola regla para las dos puertas de entrada. Como consulta la base de datos, lo envolvemos con `database_sync_to_async`, igual que en la tarea de la Clase 9. (Esa tarea queda cubierta: si el proyecto no existe, nadie es miembro, y la conexión se rechaza.)
- `self.joined = True` marca que la conexión entró al grupo.
- Al conectar, además de unirse al grupo, el consumer **avisa a todo el grupo** (incluida la propia conexión) que este usuario pasó a estar `online`; al desconectar, avisa lo contrario. `presence_update` es un método nuevo con el mismo patrón que `task_notification`: su nombre coincide con el `"type"` que usamos en `group_send`.
- En `disconnect`, `getattr(self, "joined", False)` devuelve `False` si el atributo nunca se creó. Es necesario porque Channels llama a `disconnect` **también** cuando rechazamos la conexión en `connect`, y en ese caso no hay que avisar que alguien se fue (nunca entró).

3.4 Prueba end-to-end

Necesitamos dos usuarios que sean miembros del mismo proyecto (Clase 7): por ejemplo, `ana` (admin) y `beto` (miembro) en el proyecto con `id` 1. Obtengan un `access` token para cada uno con `/api/token/` (Clase 6).

Creen, **fuera** de `taskflow/` (es una herramienta de prueba, no parte del proyecto), un archivo `escuchar.py`:

```
import sys
import websocket

proyecto, token = sys.argv[1], sys.argv[2]
ws = websocket.create_connection(f"ws://127.0.0.1:8000/ws/projects/{proyecto}/?token={token}")
print("Conectado. Esperando mensajes (Ctrl+C para salir)...")
while True:
    print(ws.recv())
```

`sys.argv` es la lista de argumentos que se escriben después del nombre del script en la terminal. Con el servidor corriendo:

1. En una terminal: `python escuchar.py 1 <TOKEN_DE_ANA>`. Aparece `{"user": "ana", "status": "online"}`.
2. En otra terminal: `python escuchar.py 1 <TOKEN_DE_BETO>`. Las **dos** terminales muestran `{"user": "beto", "status": "online"}`.
3. Desde una tercera terminal, creen una tarea como Ana:

```
curl -X POST http://127.0.0.1:8000/api/tasks/ \
  -H "Authorization: Bearer <TOKEN_DE_ANA>" \
  -H "Content-Type: application/json" \
  -d '{"project": 1, "title": "Probar tiempo real"}'
```

4. Verifiquen que **ambas** terminales reciben `{"event": "task_created", "task_id": ..., "title": "Probar tiempo real"}` instantáneamente, sin hacer ninguna petición HTTP.
5. Cierren la terminal de Beto con `Ctrl+C`: la de Ana muestra `{"user": "beto", "status": "offline"}`.
6. Prueben conectarse **sin** token, o con el token de un usuario que no es miembro: `escuchar.py` termina con un error de handshake. Es el rechazo que programamos.

3.5 Reconexión automática y sincronización (lado cliente)

Concepto. Las conexiones persistentes se cortan: se cae el wifi, el celular cambia de red, el servidor se reinicia en un despliegue, o la plataforma corta las conexiones largas (Vercel lo hace a los 5 minutos). La reconexión es responsabilidad del **cliente** (el código que corre en el navegador o la app), y tiene tres partes:

- **Reintentar con espera creciente** (*backoff exponencial*): 1 segundo, después 2, 4, 8... hasta un tope. Reintentar enseguida y sin pausa saturaría al servidor justo cuando está en problemas.
- **Agregar azar** (*jitter*): si el servidor se reinicia con 1.000 clientes conectados, sin azar todos reintentarían en el mismo instante. Sumar unos milisegundos al azar reparte la carga.
- **Resincronizar:** los avisos son "como máximo una vez" (Parte 1.2): lo que pasó mientras estaba desconectado, se perdió. Al reconectar, el cliente vuelve a pedir el estado por la API REST (por ejemplo, `GET /api/tasks/`). Esto es la **sincronización de datos en tiempo real** del programa: el WebSocket avisa rápido, la API REST garantiza que los datos sean correctos.

Como el `access` token vence a los 30 minutos, antes de reconectar el cliente también tiene que renovarlo con `/api/token/refresh/` (Clase 6); por eso el código recibe una **función** que devuelve el token vigente, no el token fijo.

Práctica (para el frontend de la Clase 13 o cualquier cliente JavaScript).

```
// Se conecta al WebSocket de un proyecto y reconecta solo si la conexión se corta.
function conectarProyecto({ url, obtenerToken, alRecibir, alReconectar }) {
  let espera = 1000;          // primer reintento: 1 segundo
  let socket;
  let cerradoAPropósito = false;

  function abrir() {
    socket = new WebSocket(`${url}?token=${obtenerToken()}`);

    socket.onopen = () => {
      espera = 1000;          // conexión exitosa: se reinicia la espera
      alReconectar();         // resincronizar: pedir por REST lo que se pudo perder
    };

    socket.onmessage = (evento) => alRecibir(JSON.parse(evento.data));

    socket.onclose = () => {
      if (cerradoAPropósito) return;
      const demora = espera + Math.random() * 500;   // "jitter": un poco de azar
      console.log(`Conexión perdida. Reintento en ${Math.round(demora)} ms`);
      setTimeout(abrir, demora);
      espera = Math.min(espera * 2, 30000);          // 1s, 2s, 4s, 8s... hasta 30s
    };
  }

  abrir();
  return () => { cerradoAPropósito = true; socket.close(1000); };
}
```

Uso:

```
const cerrar = conectarProyecto({
  url: "ws://127.0.0.1:8000/ws/projects/1/",
  obtenerToken: () => accessToken,
  alRecibir: (mensaje) => console.log(mensaje),
  alReconectar: () => loadTasks(),   // la función de la Clase 13 que pide las tareas por REST
});
```

- `onclose` se ejecuta con cualquier corte. Si lo cerramos nosotros (`cerrar()`), la variable `cerradoAPropósito` evita que reintente.
- `Math.min(espera * 2, 30000)` duplica la espera en cada falla, con un tope de 30 segundos.
- `alReconectar` se llama en **cada** apertura, también la primera: así la pantalla siempre arranca con datos frescos.

Este código se probó cortando y volviendo a levantar el servidor: reintentó a los 1, 2 y 4 segundos, y se reconectó y resincronizó solo cuando el servidor volvió.

Parte 4 — Guardar el avance en Git y GitHub (20 min)

```
git status
git add .
git commit -m "Agrega Redis como channel layer, broadcast de eventos, WebSocket autenticado con JWT y
presencia"
git push
```

Parte 5 — Cierre y tarea

5.1 Resumen — cierre de la Unidad 4

- Redis como channel layer compartido entre procesos: base necesaria para escalar horizontalmente.
- Patrón Productor/Broker/Consumidor aplicado concretamente con `group_send` y `"type"`, desde un servicio.
- WebSocket protegido con las mismas reglas que la API REST: JWT (middleware propio) y membresía al proyecto.
- Notificaciones de eventos de negocio (tarea creada) y presencia de usuarios online/offline sobre el mismo mecanismo de grupos.

5.2 Tarea para la casa

- Agreguen un evento `task_updated` (al editar una tarea, en `perform_update` del ViewSet) y `task_completed` (al usar la acción `complete` de la Clase 5), siguiendo el mismo patrón que `notify_task_created` en `NotificationService` (reutilicen `_send`).
- Respondan por escrito la pregunta 3 del cuestionario (Redis vs. Kafka) con sus propias conclusiones, a partir de lo trabajado hoy.
- Guarden el avance repitiendo el flujo de la Parte 4.

5.3 Próxima clase — Unidad 5

Clase 11: agregamos una **capa GraphQL** sobre los mismos modelos, para comparar el enfoque contra REST.

Errores frecuentes

- **`ConnectionRefusedError` / `Error 111 connecting to 127.0.0.1:6379`** en la consola al conectar un WebSocket: Redis no está corriendo. Verifiquen con `redis-cli ping`.
- **El POST devuelve 201 pero nadie recibe la notificación**: en la consola del servidor aparece `No se pudo enviar la notificación en tiempo real` (Redis caído), o el nombre del grupo no coincide (`project_1` en un lado y `projects_1` en el otro), o el `"type"` no coincide con el nombre del método del consumer.
- **La conexión se rechaza siempre, incluso con token**: el `access` venció (duran 30 minutos), o el usuario no es miembro de ese proyecto (revisen en `/admin/` → **Project memberships**).
- **Aparecen avisos `offline` de usuarios que nunca llegaron a conectarse** (por ejemplo, alguien sin permiso que intentó entrar): falta el chequeo de `joined` al principio de `disconnect`.

Preguntas de repaso

1. ¿Qué diferencia hay entre un comando y un evento? Den un ejemplo de cada uno en TaskFlow.
2. Comparen publicar/suscribir con una cola de mensajes.
3. ¿Qué garantía de entrega da el channel layer de Channels? ¿Qué consecuencia tiene para el cliente?

4. ¿Qué es la consistencia eventual?
5. Comparen Redis pub/sub, RabbitMQ y Kafka.
6. ¿Qué diferencia hay entre escalar vertical y horizontalmente? ¿Por qué el layer en memoria impide escalar horizontalmente?
7. ¿Por qué un evento debe publicarse después de confirmar la transacción?
8. ¿Por qué los WebSockets no están protegidos por CORS? ¿Qué es el *Cross-Site WebSocket Hijacking* y por qué nuestro esquema no es vulnerable?
9. Nombren dos casos difíciles de un sistema de presencia y cómo se resuelven.
10. Expliquen las tres partes de una buena reconexión: backoff, jitter y resincronización.

Para profundizar

- Documentación de Channels sobre channel layers: `channels.readthedocs.io/en/latest/topics/channel_layers.html`
- Documentación de Redis sobre pub/sub: `redis.io/docs/latest/develop/interact/pubsub/`
- Martin Fowler, *What do you mean by "Event-Driven"?*: `martinfowler.com/articles/201701-event-driven.html`
- OWASP, *WebSocket Security Cheat Sheet*: `cheatsheetseries.owasp.org`

Diccionario de términos de esta clase

- **Backoff exponencial:** estrategia de reintento que espera cada vez más tiempo entre intentos fallidos (1s, 2s, 4s...), para no saturar al servidor.
- **Broadcast:** envío de un mismo mensaje a todos los suscriptos de un canal o grupo.
- **Broker:** componente intermediario (Redis, en nuestro caso) que recibe eventos de productores y los distribuye a consumidores.
- **Channel layer:** el mecanismo de Django Channels que permite comunicar distintas conexiones y procesos entre sí, mediante grupos.
- **Comando:** mensaje que pide que algo ocurra; tiene destinatario y puede ser rechazado.
- **Consistencia eventual:** propiedad de un sistema en el que los datos se ponen de acuerdo un tiempo después, no al instante.
- **EDA (Event-Driven Architecture, arquitectura orientada a eventos):** patrón en el que los componentes se comunican publicando y reaccionando a eventos, en vez de llamarse directamente entre sí.
- **Escalado horizontal:** sumar más copias de una aplicación detrás de un balanceador de carga.
- **Evento:** mensaje que informa que algo ya ocurrió; se nombra en pasado.
- **Garantía de entrega:** promesa de un sistema de mensajería sobre si un mensaje puede perderse o repetirse.
- **Grupo (group):** conjunto de conexiones WebSocket identificadas por un nombre común, a las que se les puede enviar un mensaje simultáneamente.
- **Heartbeat (latido):** señal periódica que confirma que una conexión sigue viva.
- **Jitter:** pequeña demora al azar que se suma a los reintentos para que no ocurran todos a la vez.
- **Middleware de Channels:** capa que procesa una conexión WebSocket antes de que llegue al consumer (por ejemplo, para identificar al usuario).
- **Presencia:** funcionalidad que informa qué usuarios están conectados en un momento dado.
- **Pub/sub (publicar/suscribir):** modelo en el que cada mensaje llega a todos los suscriptos.
- **Query string:** parte de una URL después del signo `?`, con parámetros `clave=valor`.
- **Redis:** base de datos en memoria, de código abierto, usada acá como channel layer compartido entre procesos.